

Assignment-2

Student: Ashutosh Ravi Gupta

Roll: P25CS0002

Github: [link](#)

Problem 1 — Learning word embeddings from IIT Jodhpur data

Objective:

Train **Word2Vec** models on text collected from **IIT Jodhpur** sources and analyse the **semantic structure** captured by the learned embeddings.

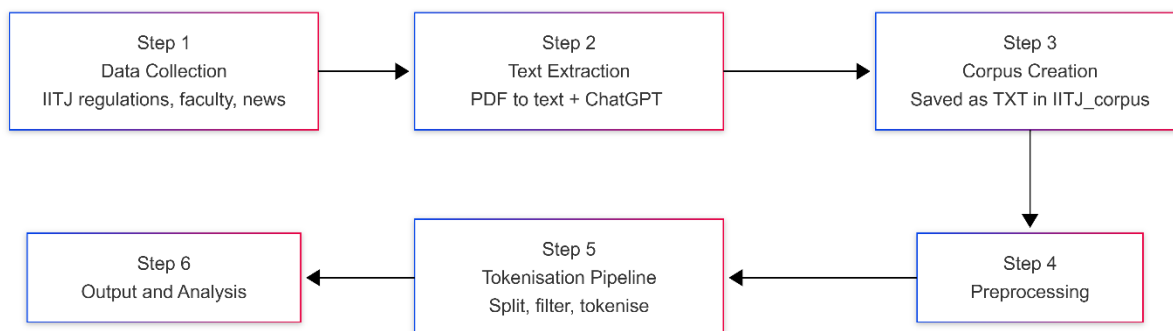
Primary artefacts in Problem 1 submission:

Artefact	Location
Source notebook	Problem1_IITJ.ipynb
Detailed technical write-up	Report.pdf
Cleaned corpus	PA1_Task1_output/clean_corpus.txt
Corpus statistics (JSON)	PA1_Task1_output/corpus_statistics.json
Preprocessing pipeline script	PA1_Task1.py

TASK-1: Dataset preparation

Assignment (summary)

- Collect English text, I have selected **academic regulations** (compulsory), **newsletters** and **faculty details**.
- **Remove non-English** text where it appears.
- **Preprocess:** (i) boilerplate / formatting noise, (ii) tokenisation, (iii) lowercasing, (iv) punctuation / non-text cleanup.



My Data extraction pipeline

Steps	What we did
Step-1	Gathered IIT Jodhpur-related material. I have selected academic regulations (compulsory), newsletters and faculty details .
Step-2	- Extracted text page-by-page (e.g. from PDFs); - used ChatGPT with prompts for plain English, Word2Vec-friendly lines.
Step-3	Saved merged extracts as .txt under IITJ_corpus/.
Step-4	Ran PA1_Task1.py : - Boilerplate removal (URLs, cookies, etc.), - lowercasing, - punctuation normalisation, - paragraph split, - short-chunk filtering, - tokenisation clean_corpus.txt + corpus_statistics.json .
Step-5	Problem1_IITJ.ipynb loads the clean corpus for stats, word cloud (content words), and all Word2Vec experiments.

Dataset statistics

Metric	Value
Total documents (source files)	3
Total lines / chunks in clean corpus	1,340
Total tokens (whitespace split)	27,299
Vocabulary size (all tokens)	3,778
Content tokens (for word cloud filters)	19,713
Content vocabulary	3,583

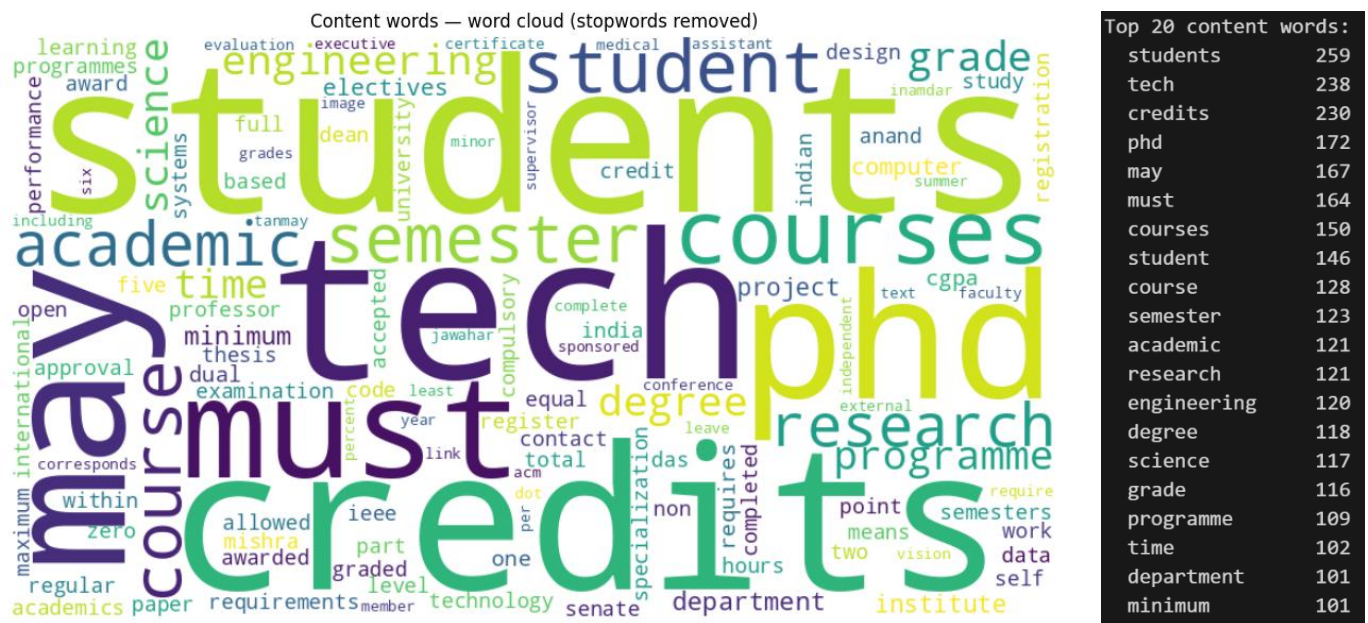


Figure P1-1 — Word cloud (most frequent *content* words)

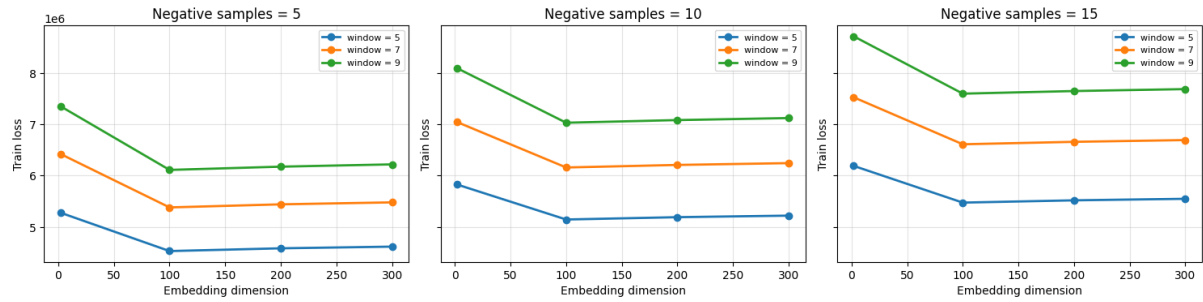
TASK-2: Model training

Assignment (summary)

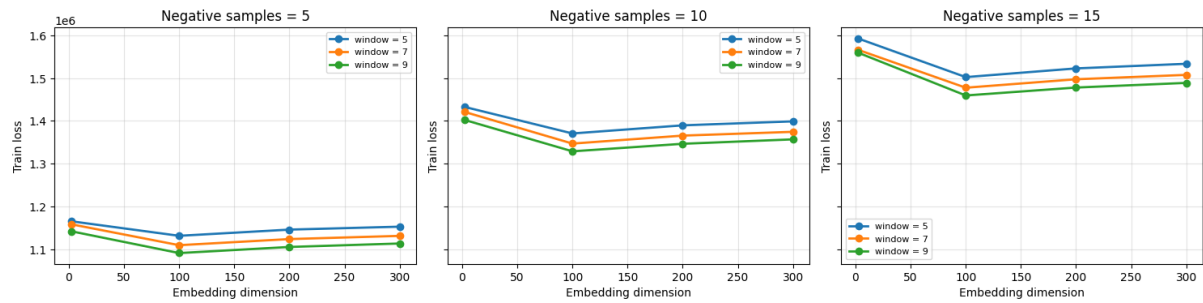
- Train **CBOW** and **Skip-gram** with **negative sampling**.
- Vary formally: **embedding dimension**, **context window**, **number of negative samples**.
- Present results **formally** in the report (tables + discussion).

Hyperparameter Analysis

Skip-gram — loss vs embedding size (lines = window size)



CBOW — loss vs embedding size (lines = window size)

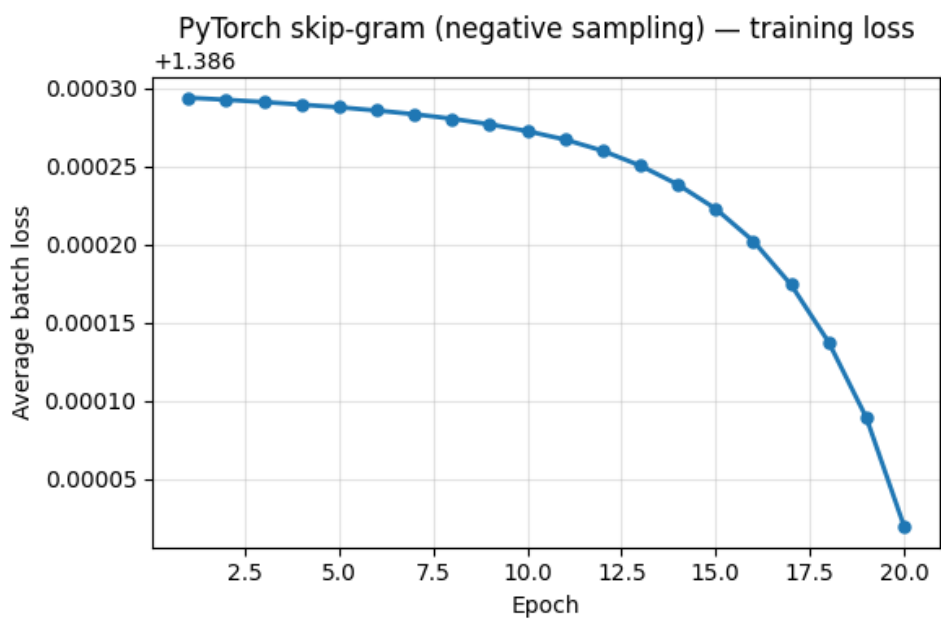


Rank	Embedding Dim	Window	Negative	Train Loss
1	100	9	5	1,091,053
2	200	9	5	1,105,345
3	100	7	5	1,109,483
4	300	9	5	1,113,456
5	200	7	5	1,123,790
6	300	7	5	1,131,249
7	100	5	5	1,131,429
8	2	9	5	1,142,104
9	200	5	5	1,145,917
10	300	5	5	1,152,779
11	2	7	5	1,158,477
12	2	5	5	1,165,444
13	100	9	10	1,328,924
14	200	9	10	1,346,578
15	100	7	10	1,347,034
16	300	9	10	1,356,910
17	200	7	10	1,365,681
18	100	5	10	1,370,711
19	300	7	10	1,374,550
20	200	5	10	1,389,683
21	300	5	10	1,399,024
22	2	9	10	1,402,461
23	2	7	10	1,421,162
24	2	5	10	1,432,872
25	100	9	15	1,459,511
26	100	7	15	1,477,673
27	200	9	15	1,478,062
28	300	9	15	1,488,941
29	200	7	15	1,497,435
30	100	5	15	1,502,478
31	300	7	15	1,507,668
32	200	5	15	1,522,804
33	300	5	15	1,533,599
34	2	9	15	1,559,926
35	2	7	15	1,566,532
36	2	5	15	1,593,099

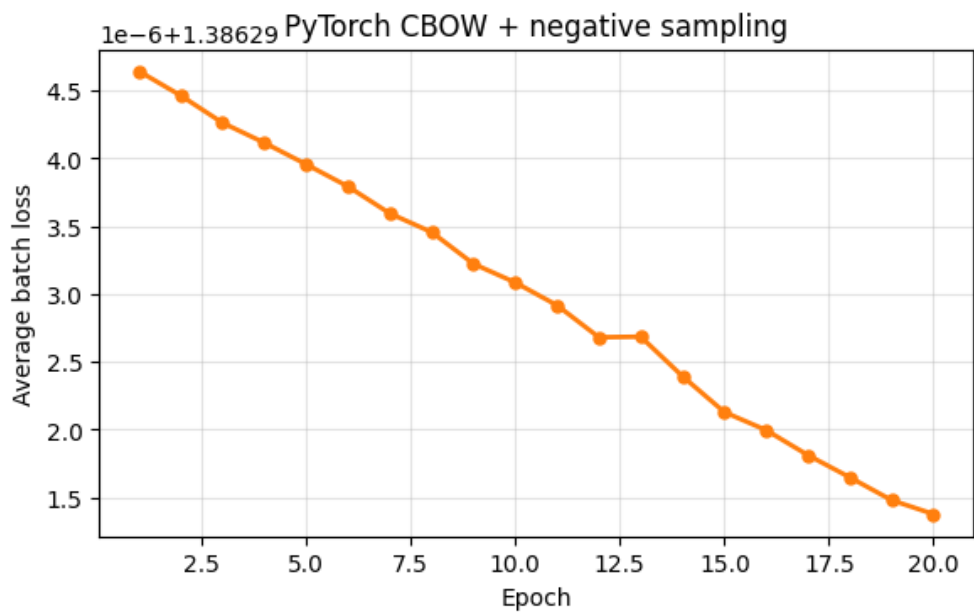
TABLE 2 —GENSIM CBOW: TRAINING LOSS VS EMBEDDING DIMENSION

Rank	Embedding Dim	Window	Negative	Train Loss
1	100	5	5	4,531,626
2	200	5	5	4,585,571
3	300	5	5	4,618,120
4	100	5	10	5,146,140
5	200	5	10	5,191,505
6	300	5	10	5,221,702
7	2	5	5	5,278,237
8	100	7	5	5,381,504
9	200	7	5	5,441,443
10	100	5	15	5,475,022
11	300	7	5	5,479,804
12	200	5	15	5,519,459
13	300	5	15	5,549,184
14	2	5	10	5,826,654
15	100	9	5	6,111,447
16	100	7	10	6,158,902
17	200	9	5	6,174,922
18	2	5	15	6,189,286
19	200	7	10	6,208,444
20	300	9	5	6,219,487
21	300	7	10	6,243,471
22	2	7	5	6,421,688
23	100	7	15	6,610,558
24	200	7	15	6,658,182
25	300	7	15	6,692,608
26	100	9	10	7,029,927
27	2	7	10	7,042,827
28	200	9	10	7,081,514
29	300	9	10	7,120,893
30	2	9	5	7,350,344
31	2	7	15	7,524,374
32	100	9	15	7,596,053
33	200	9	15	7,646,772
34	300	9	15	7,684,175
35	2	9	10	8,091,175
36	2	9	15	8,711,560

TABLE 1 —GENSIM SKIP-GRAM: TRAINING LOSS VS EMBEDDING DIMENSION



PyTorch skip-gram: average training loss per epoch



PyTorch CBow: average training loss per epoch

Model	embedding_dim	window	negative	train_loss
CBOW	100	9	5	1 091 053.125
Skip-gram	100	5	5	4 531 626.0

Best results

We trained Gensim Word2Vec in two modes—CBOW and Skip-gram with negative sampling—and searched over embedding dimension, context window, and number of negative samples.

The Above table records the best-performing runs from that grid (lowest training loss reported by Gensim with `compute_loss=True`).

- For CBOW, the lowest loss was achieved with embedding size 100, window 9, and 5 negative samples (train loss $\approx 1.09 \times 10^6$).
- For Skip-gram, the best setting was 100 dimensions, window 5, and 5 negatives (train loss $\approx 4.53 \times 10^6$).

Gensim's loss is an internal, architecture-dependent quantity, so we should not treat the CBOW and Skip-gram numbers as directly comparable; what matters is that within each architecture we picked the hyperparameters that minimised that metric, and we will use these same settings (where relevant) for later neighbour and analogy analysis.

TASK-3: Semantic analysis

Top-5 nearest neighbours (cosine similarity)

Words: **research**, **student**, **phd**, **exam**.

Gensim

Target word	Model	Neighbour 1	Neighbour 2	Neighbour 3	Neighbour 4	Neighbour 5
research	Gensim CBOW	my	dr	he	india	hyderabad
research	Gensim Skip-gram	engineer	interests	doctoral	broadly	platform
student	Gensim CBOW	cancelled	additional	if	considered	later
student	Gensim Skip-gram	withdraws	fails	cancelled	revision	considered
phd	Gensim CBOW	degree	master	mtech	switch	dual
phd	Gensim Skip-gram	tech	bachelor	master	msc	switch
exam	Gensim CBOW	applying	tuition	improvement	fee	different
exam	Gensim Skip-gram	comprehensive	semesters	candidacy	art	seminar

PyTorch

Target word	Model	Neighbour 1	Neighbour 2	Neighbour 3	Neighbour 4	Neighbour 5
research	PyTorch Skip-gram	the	of	in	by	at
research	PyTorch CBOW	reading	memberijcai	out	communication	cognitive
student	PyTorch Skip-gram	of	by	with	and	in
student	PyTorch CBOW	acceptance	scheme	clinically	mini	foundations
phd	PyTorch Skip-gram	tech	degree	credits	dual	requires
phd	PyTorch CBOW	icdar	allowed	rd	entrepreneurship	moderated
exam	PyTorch Skip-gram	icpr	still	jodhpur	gave	sc
exam	PyTorch CBOW	intersection	minister	designed	thanks	structura

Analogy experiments

1. UG : BTech :: PG : ?

- a. Gensim
 - i. Skip-gram returns words like *highlights, pursuing, membership*;
 - ii. BOW returns *kanpur, chennai, leading*.
- b. PyTorch
 - i. **Skip-gram**: *arxiv, publications*;
 - ii. **CBOW**: *intersection, mentorship*.

2. student : course :: faculty : ?

- a. Gensim
 - i. **Skip-gram**: *coordinators, offering, head* — partly fits admin/faculty language.
 - ii. **CBOW**: *means, unique, type, identification, structure* — very **generic**, not really “faculty”
- b. PyTorch
 - i. **Skip-gram**: *genus, from, science, department, technology* — department / science / technology are partially related to an institute.
 - ii. **CBOW**: *akanksha, munich, brain, ipr* — mostly noise for this relation.

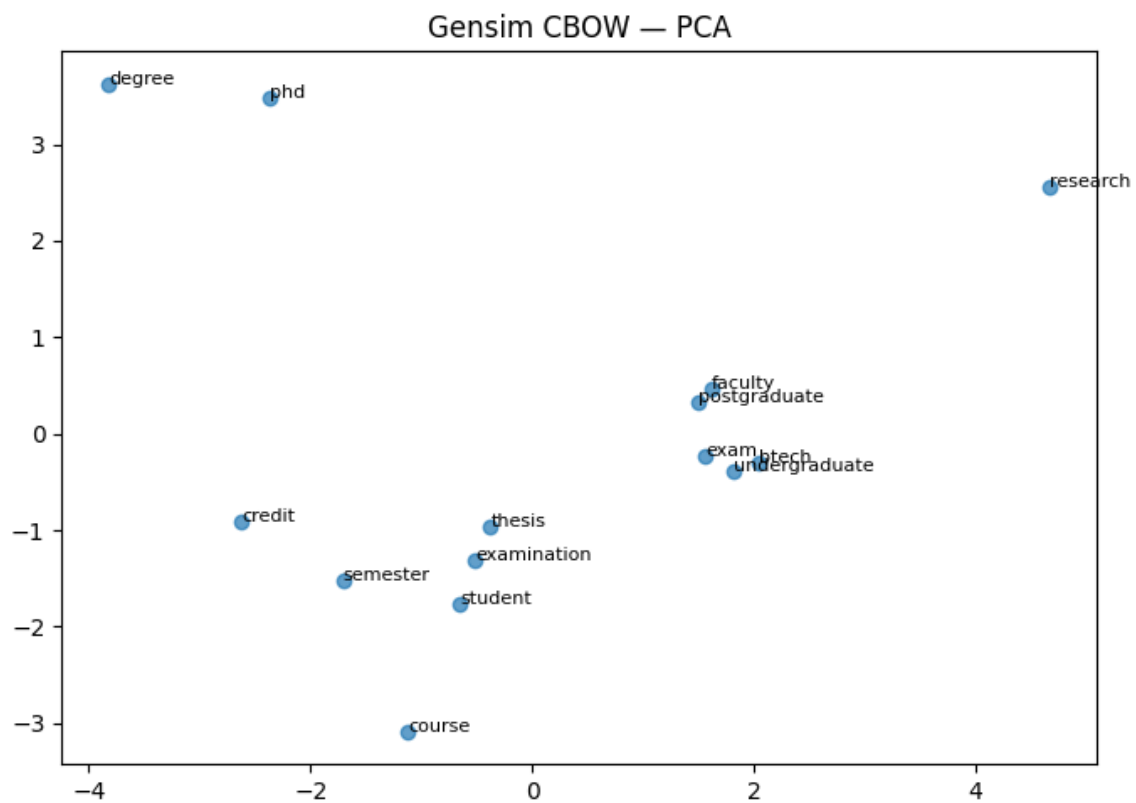
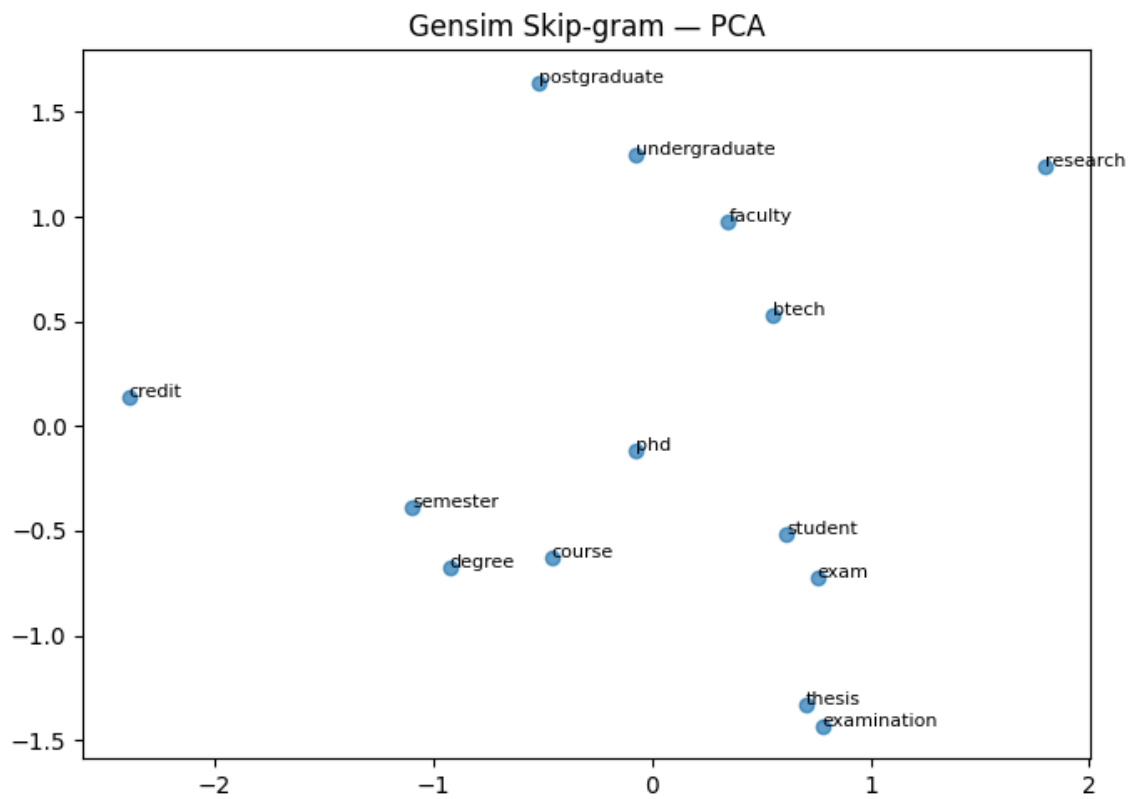
3. research : phd :: thesis : ?

- a. Gensim
 - i. **Skip-gram**: *second, thirty, earns, switch, eight* — **numbers / junk** for “thesis”.
 - ii. **CBOW**: *complete, requirement, duration, restart, required* — reads like rules / requirements, not “thesis” by name, but still policy-ish.
- b. PyTorch:
 - i. **Skip-gram**: *executive, requires, dual, degree, credits* — degree / credits are vaguely academic.
 - ii. **CBOW**: *fomin, allowed, diverse, datasets, different* — not thesis-like.

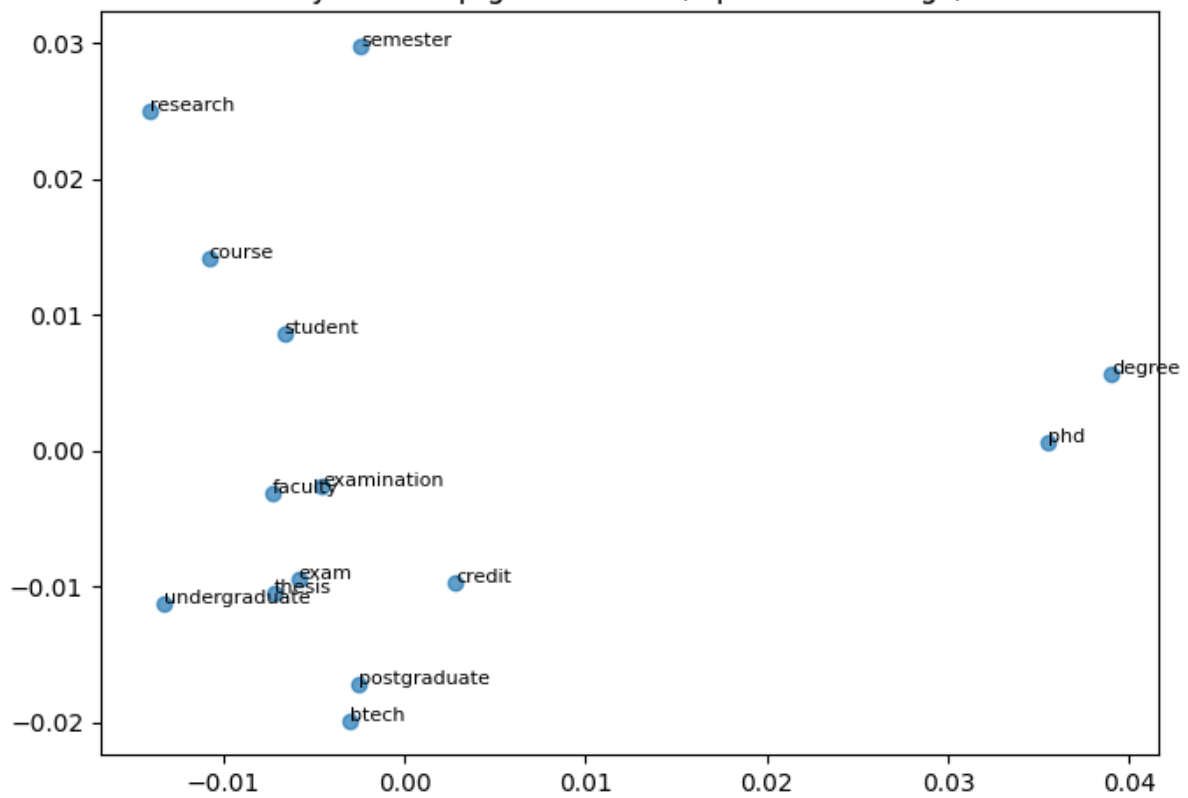
nearest-neighbour behaviour is clearest for Gensim Skip-gram (e.g. phd → degree-related types; exam → academic context), while Gensim CBOW often pulls very frequent or generic tokens (my, he, if). PyTorch neighbours are noisier: Skip-gram research/student align with stopwords, and CBOW context rows pick ragged tokens (names, conference-like strings), which is typical when training is lighter or optimisation differs from Gensim.

Analogies assume roughly linear relations in embedding space; here most Gensim and PyTorch answers are not clean “PG degree” or “thesis” substitutes— numbers and generic words dominate several Skip-gram runs. Any word not in the vocabulary after min_count filtering is OOV and cannot be used in most_similar or analogy APIs without changing preprocessing. Overall, neighbour lists support a modest semantic signal; analogy rankings should be read as weak evidence on this domain and corpus size.

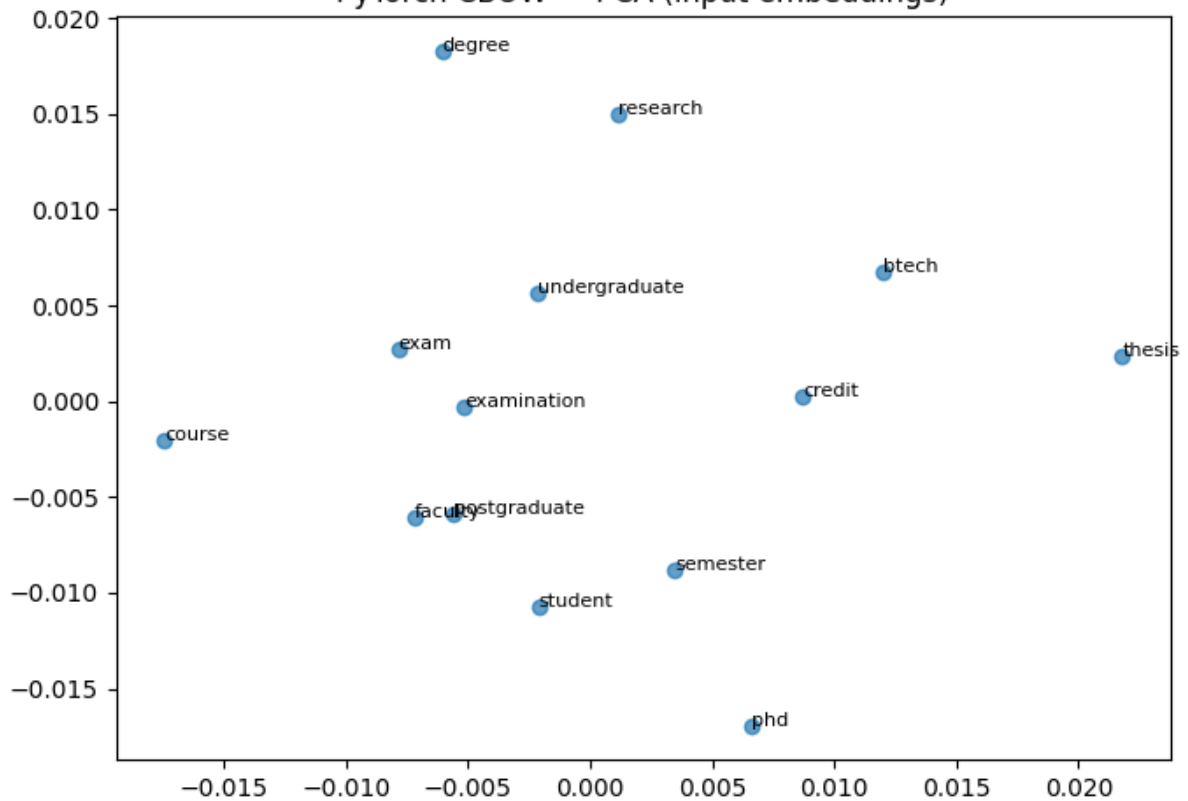
TASK-4: Visualisation (PCA or t-SNE)



PyTorch Skip-gram — PCA (input embeddings)



PyTorch CBOW — PCA (input embeddings)



Problem 2 — Character-level name generation using RNN variants

Objective:

Design and **compare** sequence models for **character-level** name generation using **recurrent** architectures.

Primary artefacts in Problem 1 submission:

Artefact	Location
Source notebook	problem2.ipynb
Detailed technical write-up	Report.pdf
Name list (LLM-generated)	TrainingNamesGPT.txt

TASK-1: Model implementation

Setting	Value
Embedding dim	64
Hidden size	128
RNN / LSTM layers	1
Optimizer	Adam
Learning rate	0.001
Batch size	64
Training epochs	35
Vocabulary size (chars + <pad> + \n)	29
Training samples (prefix → next char)	13,336

Architecture summary

1. **Vanilla RNN** — Embedding → nn.RNN (tanh) → last hidden state → Linear to vocabulary.
2. **BiLSTM (BLSTM)** — Embedding → bidirectional nn.LSTM → concatenated forward/backward at last timestep → Linear.
3. **RNN + attention** — nn.RNN over the prefix; scaled dot-product style attention over timesteps (padding masked); concatenate query and context → Linear.

Trainable parameters and vanilla RNN size

Model	Trainable parameters
Vanilla RNN	30,429
BiLSTM	207,965
RNN + Attention	50,525

Vanilla RNN model size (memory of parameters) = **0.1161 MB** (sum of numel × element_size for all tensors, float32).

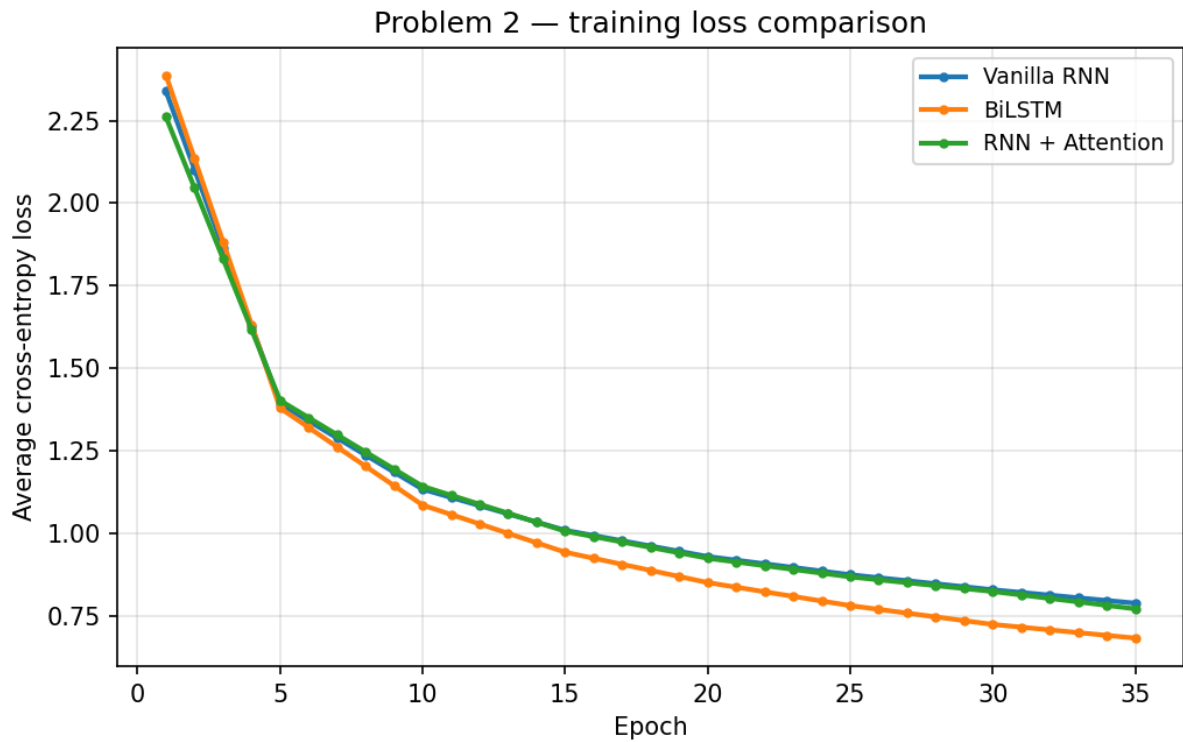
BiLSTM is largest because each LSTM layer has four gated linear transforms per direction, and two directions are stacked before the readout.

Training loss (cross-entropy, next character)

Average loss per example at selected epochs:

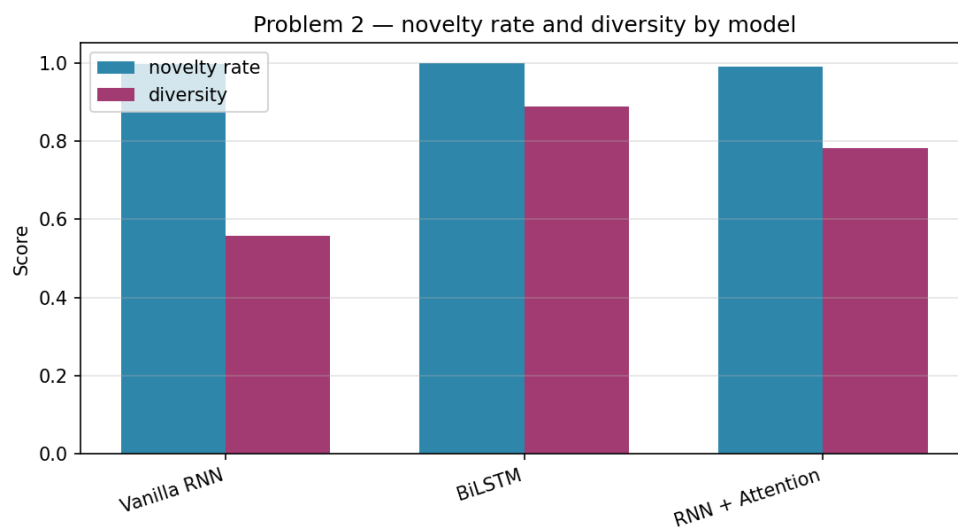
Epoch	Vanilla RNN	BiLSTM	RNN + Attention
1	2.3389	2.3875	2.2611
5	1.3920	1.3792	1.4022
10	1.1330	1.0850	1.1420
15	1.0087	0.9423	1.0059
20	0.9291	0.8502	0.9239
25	0.8740	0.7804	0.8675
30	0.8284	0.7234	0.8232
35	0.7878	0.6819	0.7706

Lowest final loss: BiLSTM (**0.6819**), then RNN + Attention (**0.7706**), then Vanilla RNN (**0.7878**).



TASK-2: Quantitative evaluation

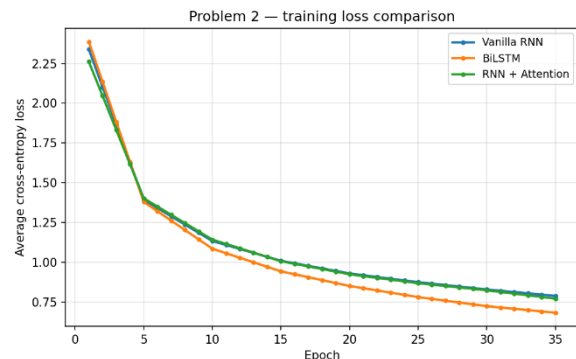
Model	Novelty rate	Diversity
Vanilla RNN	0.998	0.558
BiLSTM	0.999	0.889
RNN + Attention	0.990	0.782



Comparison :

In my experiments, BiLSTM did the best overall: it reached the lowest training loss and also had the highest diversity, while novelty stayed at 1.0 in my table—so almost everything I generated was not an exact copy of a training name. RNN + Attention was clearly better than the vanilla RNN on diversity and loss, but it did not beat the BiLSTM. My vanilla RNN was the smallest model and still had very high novelty, but diversity was much lower, which matched what I saw when reading samples: it tended to reuse similar strings more often.

I think BiLSTM helps on a small name corpus because names are short but still follow patterns (letters, syllables, spaces). The LSTM handles long-range dependencies in the prefix better than a plain RNN, and having a backward direction gives the model extra context about the whole prefix before predicting the next character. That seems to improve fitting the data (lower loss) and makes sampling less repetitive (higher diversity), without collapsing into memorization.



TASK-3: Qualitative analysis

Realism and failure modes

When I read the generated names by eye, **spacing** was one of the first things I noticed: sometimes the model produced **no space** between what should be two parts of a name, or **awkward gaps**, because it only learns from characters, not from “first name / last name” rules.

Indian name patterns looked reasonable when the model copied bigrams and rhythm from the training file—lots of samples **felt** like names from the same style—but other times they looked **generic** or had **unusual letter combinations** that a person probably wouldn’t use.

I also saw **too-short outputs** (sometimes just a couple of letters before the model stopped), and occasional **gibberish** or **repeated chunks** when the uncertainty was high or sampling went off-track.

Even when **novelty** was high, I still found **near-duplicates**: names that were **not exactly** in the training list but were **very close** to one training example—same shape, one letter different—which makes sense because the model stays near the training distribution.

Representative samples:

Vanilla RNN:

hanna, lanan, redey, redha, ghetam, er, bhandar, bharna, anni, achord, luk taik, man, ti, ofi, jandiy, dhy, ukh, i, hetti khatala, purik gi.

BiLSTM:

ga dwajoj choudhry, dh kurya, shul menon, riy, tej joshi, awat, hanrey ahmed, arg yadav, dha joshi, y, minen, sha mehta, meher naj, dhy, kabhra bharge, i bhatia, them ghosh, vya choudhury, kshant chatterjee, hee.

RNN + Attention:

ree reddy, ni ram, ur jali, ill bhatia, shi varma, dev kapoor, khel opnachary, ya khan, a dubey, tej choudhary, bhita puryap, ali patil, a kapoor, udha dutta, rohit patel, ali sinha, rita verma, an khan, bhata sundaram, al oppoor.